DOINGG@UNION.EDU

COURSE: CSC106 CAN COMPUTERS THINK?

SELF

Contents

Home

The course website for CSC106, part of the Union College CS curriculum.
Here you can find our weekly schedule, assignmnets and resources.

Announcements

Project 1 due this Friday (4/17/26)

- 10% of total course grade
- will need to use Python documentation
- will be easier after doing texbook reading/exercises

Practical Exam next Thursday (4/23/26)

- 1 hr
- live coding **and** some written questions
- 10% of total course grade
- be sure to be caught up on texbook reading/exercises

Office Hours

- Student/Office Hours:
  - Steinmetz 108B
  - Monday 3:00 pm - 4:00 pm
  - Thursday 2:00 pm - 4:00 pm
  - subject to change, check course website for most up-to-date schedule
  - drop-in or schedule a 15 minute slot: https://calendar.app.google/8bus6pfDvyphR9ar5
  - by appointment for another time or over zoom

Course Description

Can computers think? Is it possible to build intelligent machines? What does it mean for a machine to think or to be intelligent? Why, or why not, would we want intelligent machines? To what extent do we already have them? How do they work? How do they impact our lives? What are their limitations? This course invites you to explore these questions and more.

This course is an introduction to the field of computer science with an artificial intelligence theme that does not assume you have prior experience with computer programming or computer science. It introduces basic algorithms, data structures and programming techniques. You will learn how computer scientists think about and solve problems by doing your own computer programming. This will give you an understanding of how the current technology for building intelligent machines works. In addition, you will read non-technical papers about artificial intelligence and the ethical questions raised by its use. You will reflect on the ways current computational systems often reinforce biases and can harm groups of people inequitably. And you will evaluate how the choices you are making when creating your own programs will affect future users.

If you're thinking about a CS or CPE major or minor, this course will give you a solid foundation to continue to *Programming on Purpose* (CSC 120),

to *Data Structures* (CSC 151) and beyond. If you're a neuroscience major, it will help you see how scientists are using biological and neurological principles to model "behavior'' in computers and it will teach you skills useful for processing experimental data. If you are thinking about another major or minor, this course will give you foundations to apply computing in whatever area you are interested in. And if you're here just because you're curious, that's great!

Schedule

Weekly Schedule

Schedule

check often as things may change

W	TOPIC	Due	Runestone
1	Programming, Calling Functions	Introductory Survey	2.1 - 2.13
2	Loops, Variables and Assignment	HW 1	3.1-3.5
3	Defining Functions	HW 2, Project 1	5.1-5.12
4	Conditional Statements	HW 3, Practical Exam 1	4.1-4.9
5	Conditional Loops	HW 4, Project 2	6.1-6.7
6	Strings, Lists	HW 5, Written Exam 1	7.1-7.12
7	Nested Lists	HW 6, Project 3	9.1-9.14
8	File I/O, Dictionaries	HW 7, Practical Exam 2	8.1-8.9
9	Dictionaries	HW 8, Project 4	10.1-10.6
10	Searching, Sorting, Recursion	HW 9, Written Exam 2	

Resources

CS-Ticket

- technical help with lab machines
- https://csticket.union.edu/

Runestone

- https://runestone.academy/runestone/default/user/register
- your Union College email
- unioncollege_py4e-int_spring26 as the course name

Gradescope

- https://www.gradescope.com/courses/1288185
- Entry Code: ZJ8ZRW

Late Token Form

- https://forms.gle/WXW44b2AM6kQCdX16
- fill out this form to get a 24 hour extension on any Programming Project
- each student gets 3 tokens per term

Python

- Python 3.13.X: the software package needed to write and run Python programs
- https://www.anaconda.com/download/success?reg=skipped

Python Documentation

- Official Documentaion: https://docs.python.org/3/index.html
- Can *always* be used as a reference
- Can be searched for module-specific documentation e.g. turtle

Reeborg

- Reeborg's World: a useful website for learning some programming basics
- Reeborg's World: https://reeborg.ca/reeborg.html
- Reeborg Project Documentation

PythonTutor

- Python Tutor: a useful website for tracing python code
- https://pythontutor.com/visualize.html#mode=edit

Google Colab Pro

- Pro account sign-up: https://colab.research.google.com/signup
- Use your Union email
- A pro account is not needed for this course, just an option Union pays for
- Notes template: https://colab.research.google.com/drive/15rPmKyQKCCbwsdDCpC39o8hczWZ-B5zV?usp=sharing

Python Style Guide

- PEP 8
- should be used in all code cells of Colab notebooks
- Link: https://peps.python.org/pep-0008/

Grading

Table 2: Grading Scheme

Course Element	Grade Point Contribution	Notes
Engagement	10%	written answers collected and scanned, 0.25% per activity: itemized list
Programming Projects	40%	4 at 10% each
Midterm	15%	written
Practical Exams	20%	coding & written
Final Exam	15%	written
total	100%	

Table 3: Letter Grade Scale

Letter Grade	Percent range
A	93 - 100
A-	90 - 92
B+	87 - 89
B	83 - 86
B-	80 - 82
C+	77 - 79

Letter Grade	Percent range
C	73 – 76
C-	70 – 72
D	60 – 69
F	0 – 59

Note: You must get a C- or better in order to take any other class that has a CSC-10x prerequisite. Several larger programming projects will be assigned to reinforce the concepts discussed in class and put into practice in homework assignments. All projects can be completed using programming techniques that have been covered in class.

Do not use techniques that have not been covered in class unless specifically told otherwise. Part of the challenge for each programming project is figuring out how to complete the assignment using only the tools provided in the course up to that point.

Assignments

HW 01

CSC106: Assignment 1 (0.25 pts)

- due Monday 4/6/26 by class time (1:50 pm)
- in-class notes from Friday: https://drive.google.com/file/d/1UW3fMha6Ki4dDR_sidpeiV-PDeBSFEPi/view?usp=sharing
  - examples of comments, importing turtle, variables and for loops
  - also includes link to turtle library documentation

Part 1

- Intro Survey
- Runestone Chapters 1-3 (we'll discuss in class next week starting Monday)

Part 2

- Practice Problems for Week 1: HW01 instructions
- Starter code: winding_sandbar.json
- Submit code on Gradescope
- bring written answers to class (0.25 engagement points awarded on these)
- Graded on effort/completion

HW 02

CSC106: Assignment 2 (0.25 pts)

- due Monday 4/13/26 by class time (1:50 pm)
- Runestone Chapter 5 (see schedule for specific sub-sections)
- Practice Problems for Week 2:
  - Submit code on Gradescope,
  - Bring written answers to class (0.25 engagement points awarded on these)
  - Graded on effort/completion

NAME:

Practice Problems for Week 2

Please complete, bring to class (written answers) and submit on gradescope (code) by end of day on **Monday 4/13/26**.

You will be submitting the following files:

- square_chain.py
- retirement.py
- *All the programming files **MUST** be named as instructed*

Programming Part 1: Turtle Stairs

1. Create a file called square_chain.py for your code.
2. Write a Python program using turtle to draw something that draws a diagonal chain of 10 squares:

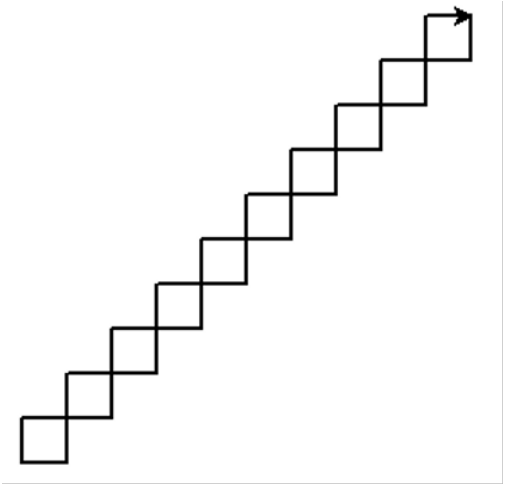

Figure 1: Turtle Stairs

There are many different ways (algorithms) that you can draw this pattern. Some may (or may not) leave the turtle in the same position and orientation that you see here. It does not matter if the turtle has the same position and orientation as what you see here.

3. Add comments to explain how your program works.
    - Header comment with Name, Email, Date, Assignment, Description
    - In-line comments describing the algorithm guiding your program
    - Docstrings for any functions, if you define new functions

Programming Part 2: Retirement Calculator

It's never too early to start saving for retirement. For this assignment you're going to write a simple retirement calculator.

- A run of the program may look like the following (user input in bold):

```
How much do you plan to save each month? (in dollars) **100**
How old are you? (in years) **20**
At what age do you plan to retire? (in years) **40**
What yearly rate of return do you expect? (in percentage points) **5**
Your savings at age 21 are: $ 1200
Your savings at age 22 are: $ 2460
Your savings at age 23 are: $ 3783
Your savings at age 24 are: $ 5172
Your savings at age 25 are: $ 6630
Your savings at age 26 are: $ 8162
Your savings at age 27 are: $ 9770
Your savings at age 28 are: $ 11458
Your savings at age 29 are: $ 13231
Your savings at age 30 are: $ 15093
Your savings at age 31 are: $ 17048
Your savings at age 32 are: $ 19100
Your savings at age 33 are: $ 21255
Your savings at age 34 are: $ 23518
Your savings at age 35 are: $ 25894
Your savings at age 36 are: $ 28388
Your savings at age 37 are: $ 31008
Your savings at age 38 are: $ 33758
Your savings at age 39 are: $ 36646
Your savings at age 40 are: $ 39679
```

1. Start by creating a file called retirement.py for your code.
2. Next, start writing code to get input from the user and store the information in variables.

- The program should start by asking the user for the following information:
  - The amount they plan to save each month
  - Their starting age
  - Their planned retirement age
  - The yearly rate of return they expect (i.e. growth through interest or dividends).

3. Now design an algorithm to compute the amount that accumulates each year. Write out the algorithm in a human language (e.g. English) before writing it in code. Include this as your description in your header comment.

- The program should show how the user's savings grow over the course of their careers. For example, if a 20 year-old starts saving $100 a month, retires at 40, and gets a 5% rate or return, then they should have about $39,679 by the time they retire.

- To simplify things, we assume that interest is added once per year before the user has contributed any of their monthly savings.
  - This means that, for the first year, the rate of return is zero, so they only get what they saved over the course of the year (i.e. $1200).
  - For subsequent years, calculate the interest gained, then add the yearly contribution. So for year 2:
    - the interest will be $1200 × 5% = $60
    - the savings at the end of year will be $1200 + $60 + $1200= $2460.

4. Implement the program in Python
5. Test the program with different values to ensure that it works correctly.
6. If you haven't already, add comments explaining how the program works.

Written Questions

1. Consider the following (incomplete) Python program:

```
x = int(input("Please input a positive integer: "))
counter = x
for step in range(0, 10):
```

Complete the missing lines of code above so that the program will produce the following output if the user inputs 3.

```
3
6
9
12
15
18
21
24
27
30
```

- **Notes**: You do not need to introduce any additional variables. There is more than one way to achieve this result.

2. Consider the following (incomplete) Python program to list the first few Fibonacci numbers:

The Fibonacci numbers are the following sequence of numbers:

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, …

- The first two numbers are 0 and 1.
- After that, each number can be calculated by adding the previous two together. For example:
  - the third number is 1 (0+1)
  - the fourth number is 2 (1+1)
  - the fifth number is 3 (2+1), etc.

They are named Fibonacci numbers after the Italian mathematician who first described them in Europe. Patterns based on the Fibonacci sequence show up in surprisingly many contexts in math, biology, and art. (See the Wikipedia entry if you want to know more.)

Below is the (incomplete) Python program to list the first few Fibonacci numbers.

The user gets to determine how many Fibonacci numbers to list.

```
1.  print ("How many Fibonacci numbers would you like to see?")
2.  limit = int (input("Please input a number greater than 2: "))
3.
4.  fib_0 = 0
5.  fib_1 = 1
6.
7.  print(fib_0)
8.  print(fib_1)
9.  for count in range(        ):
10.    next_fib =
11.    print(next_fib)
```

- For questions a-d, assume that the program has been completed and functions correctly.

2.a. Draw a memory diagram that shows the- associations after line 5 has been executed. Assume that the user inputted 10. Also show what will have been printed to the screen at that point.

2.b. Draw a memory diagram that shows the - associations that should exist after line 11 has been executed for the first time. Also show what line 11 will print to the screen when it is executed for the first time.

2.c. What should the - associations be when line 11 has been executed for the second time? Draw a memory diagram.

2.d. Draw a memory diagram that shows the - associations after the last number has been printed (i.e. at the end of the program).

2.e. Complete the code above so that the program prints the first n Fibonacci numbers, where n is determined by the user. You may need to add new lines of code.

3. Consider two Python programs:

- Both programs on the next page will help Reeborg harvest the carrots in the Harvest 1 world:

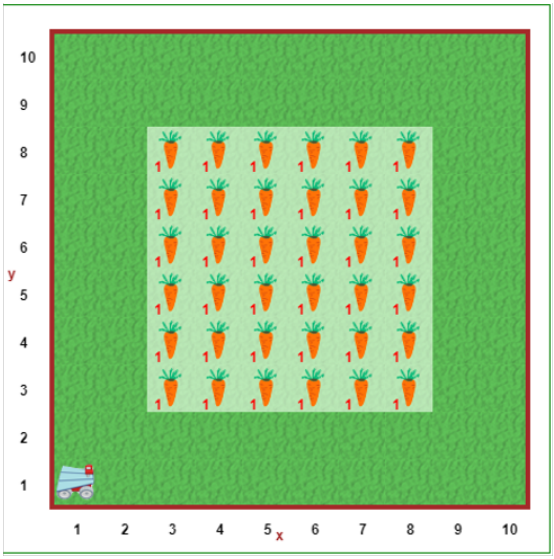

Figure 2: Reeborg Carrots

- Both programs define a number of functions. Unfortunately, the program designers were from the bad old days when names needed to be as short as possible, so their function names are essentially meaningless. Come up with better names for each of the functions that they defined.

a. .....
b. .....
c. .....
d. .....
e. .....
f. .....
g. .....
h. .....

```
# Program 1

1  def turn_right():
2   turn_left()
3   turn_left()
4   turn_left()
5
6  def a():
7   move()
8   turn_left()
9   move()
10  move()
11  turn_right()
12  move()
13
14 def b():
15  for carrot in range(0, 6):
16      take()
17      move()
18
19 def c():
20  turn_left()
21  move()
22  turn_left()
23  move()
24
25 def d():
26  turn_right()
27  move()
28  turn_right()
29  move()
30
31 a()
32 for double_row in range(0, 3):
33  b()
34  c()
35  b()
36  d()
```

```
# Program 2

1  def turn_right():
2   turn_left()
3   turn_left()
4   turn_left()
5
6  def e():
7   move()
8   turn_left()
9   move()
10  move()
11  turn_right()
12  move()
13  take()
14
15 def f():
16  for carrot in range(0, 5):
17      move()
18      take()
19
20 def g():
21  f()
22  turn_left()
23  move()
24  turn_left()
25  take()
26
27 def h():
28  f()
29  turn_right()
30  move()
31  turn_right()
32  take()
33
34 e()
35 for double_row in range(0, 2):
36  g()
37  h()
38 g()
39 f()
```

3.i Now that you've come up with better names for the functions, which of the solutions do you think is better and why?

……

……

……

……

……

……

……

……

……

……

……

……

4. Check-in

4.a. Were you able to complete all of the programming activities (yes/no)?

……

4.b. What, if anything, did you struggle with when working on the activities?

……

……

……

……

4.c. If you managed to overcome the challenges, how did you do it?

……

……

……

……

Project 1

CSC106: Project 1

- due Friday 4/17/26 by class time (1:50 pm)
- you may use late token(s) for this assignment

In this project, you'll write code to draw a variety of shapes using the turtle module. Then, you'll use this code to compose a picture and to write a program to randomly generate abstract art.

- Reminder: Programming projects should be done individually. You should not look at anyone else's code, show your code to anyone else, write code for anyone else, or let someone else write code for you. Consider anyone else to include any form of AI. Please see the syllabus or Nexus for what to do when you need help with your work.
- Projects are graded on program correctness and code composition, including proper commenting
- This project is 10% of your final grade
- starter code:
  - square.py
  - turtle_shapes.py
  - computer_art.py

NAME:

Project 1: Drawings with Turtle

In this project, you'll write code to draw a variety of shapes using the turtle module. Then, you'll use this code to compose a picture and to write a program to randomly generate abstract art.

Learning Objectives:

1. Practice designing and implementing simple algorithms.
2. Demonstrate that you can use the programming concepts that we've covered so far: function calls, function definitions, variables and assignment, and counting loops
3. Practice reading the official Python documentation.
4. More practice defining your own functions and using them.

Reminder: Programming projects should be done individually. You should not look at anyone else's code, show your code to anyone else, write code for anyone else, or let someone else write code for you. Consider anyone else to include any form of AI. Please see the syllabus or Nexus for what to do when you need help with your work.

1. Drawing shapes (4 pts)

1.1 Square

1. Open the starter code `square.py`. You'll notice that a few variables are defined at the top of the file. Please use these variables in your code.
2. Add some code that moves the turtle to the (`x`,`y`) position without drawing a line.
3. Add some code that draws the square using size as the side length.
4. Make sure to save your program in a file called `square.py`.
5. Make sure that your code is fully commented.

1.2 Rectangle

1. Create a new code file called rectangle.py using a similar format to `square.py`. Meaning:
    a. Define `x, y` to indicate the starting position for the rectangle.
    b. Create: width and height to control the size of the rectangle.
    c. The entire rectangle should be visible in the turtle window without having to resize it, but beyond this restriction, feel free to place the rectangle wherever you like and to make it as big or small as you like.
2. Add code to move the turtle to position (`x`,`y`).
3. Add code to draw a rectangle with the specified width and height.
4. Save this code in a file called `rectangle.py` and make sure to comment the code.
    - Hint: It may be very helpful to copy and paste code from `square.py` to reduce the amount of typing you have to do.

1.3 Circle

1. Follow the same general steps for the rectangle, except this time you'll be creating a circle.
    a. File name: `circle.py`.
    b. Variables to create: `x, y, radius`.

1.4 Triangle

1. Follow the same general steps as for the rectangle, but this time we're making a triangle.
    a. File name: `triangle.py`.
    b. Variables to create: `x, y, size`.
2. This program should draw an equilateral triangle.
3. The size variable should indicate the length of one side.

2, Functions for shapes (1 pt)

1. Open the starter file `turtle_shapes.py`. This file contains the beginnings of some function definitions. (We'll be discussing function definitions more this week)
2. Copy your code for drawing a square from `square.py` into the `square(x, y, size)` function definition. There are comments explaining how to do this in the starter file.
    - Caution: Do not copy the variable definitions at the top of the square.py file. They will not be needed if you designed your square.py code correctly.
3. You should notice that the `square(x, y, size)` function is called at the bottom of the starter file. Once you've completed the previous step, running the program should produce a drawing of a square in the bottom right quadrant of the turtle window.
4. Repeat step 2 for the other shapes (rectangle, circle, triangle).
    - Caution: Don't change the order of the parameters in the function definitions.
5. Add function calls for each of the shapes at the bottom of the file to test your program and prove that the functions work as expected.
6. Please note that you should copy over comments in addition to the code itself.

3. Draw a picture (2 pts)

1. Create a new file and copy the function definitions from Part 2 into it. Save this new file as `my_picture.py`.
2. Write a program that uses these functions to draw a picture.
3. Picture requirements:
    a. Use at least two line colors and at least two fill colors.
    - Hint: it may be useful to refer back to your in-class assignments. If you're still having trouble, take a look at the `turtle` library section of the Python official documentation (https://docs.python.org/3/library/turtle.html)… or stop by my office hours.
    b. Your picture should depict a recognizable object or scene. For example, something like the picture bloew (though it does not need to be this involved)

c. You may add additional functions to help draw the picture (e.g. functions to draw filled-in versions of each shape), but you should not change the functions that you defined for Part 2 of the assignment.
d. Your program should use each function from Part 2 at least once, but you can also use other built-in turtle functions.
e. Your program should use loops when appropriate to keep your code DRY (Don't Repeat Yourself).

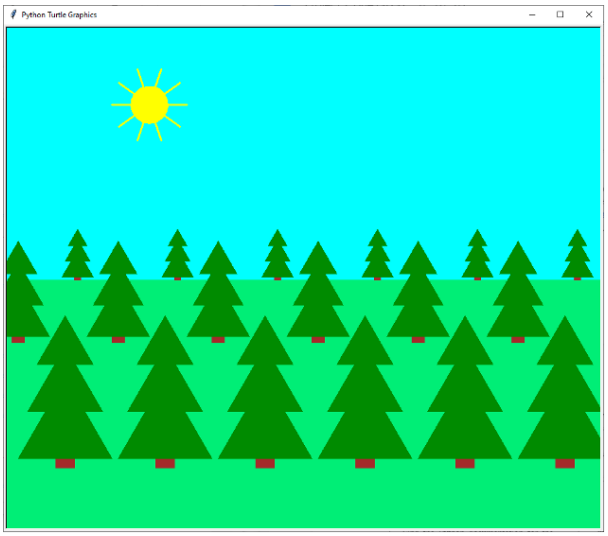

Figure 1: Turtle Drawing

4. Computer Generated art (3 pts)

1. Find the Python documentation for the `random` module. (Hint: Use the link to the "library reference" on the Resources page on the course website)
2. The function `randrange` from the `random` module produces a random integer. Test it out by writing a program that generates five random numbers between 2 and 9 (inclusive) and prints them out. Save your program in a file called `five_random_numbers.py`. Hint: remember to keep your code DRY.
3. Open the starter file `computer_art.py`.
4. This file already contains definitions for two functions. In particular, it contains a function called `random_shape` that randomly picks a shape (square, rectangle, circle, or triangle) and draws it. The function takes three parameters:

    a. `x` - the x location
    b. `y` - the y location
    c. `size` - the size

5. Copy and paste your function definitions from Part 2 into this file.
6. Add some code to draw 30 randomly picked shapes at random locations on the screen. For example, your output could look something like this:

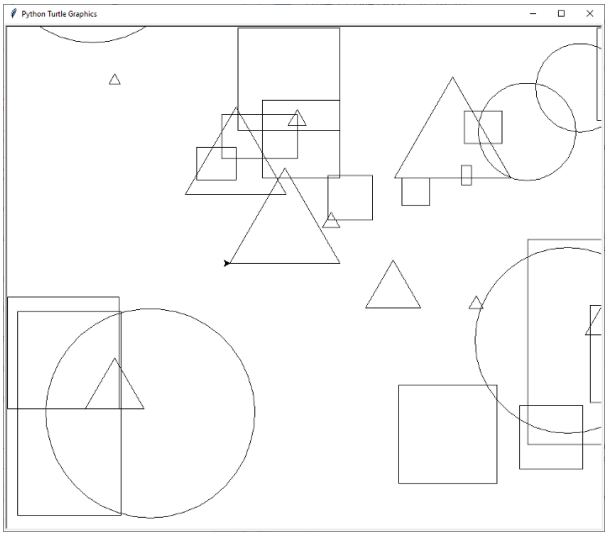

Figure 2: Computer Art

7. (Optional): Add some code to randomly change the pensize, pencolor, and filcolor.

5. What to submit

You will need to submit the following files:

1. `square.py`
2. `rectangle.py`
3. `circle.py`
4. `triangle.py`
5. `turtle_shapes.py`
6. `my_picture.py`
7. `five_random_numbers.py`
8. `computer_art.py`

Before submitting, check that your code meets the following requirements:

1. Are your files properly commented? This should include the following:
    a. A header comment, which includes your name, the date, the assignment and a brief description of the program.
    b. Comments within the body of the code to further clarify how the program works.
2. Are your programs formatted neatly and consistently? Do you use some whitespace to help a human reader understand the logical organization of your code?
    - Hint: it may be helpful to think of this as breaking your code up into "paragraphs."
3. Have you cleaned up any code snippets you no longer need?
    a. The final version of the program should not contain any lines of actual code that are commented out to keep them from running. Such snippets should be removed before submitting.

Once you have checked these things, submit your code to:

"cat3_Project 1: Drawings with Turtle" on Gradescope.

6. Grading guidelines

- Correctness: programs do what the problems require.
- Program logic: use loops where appropriate, variables where appropriate, and your code doesn't contain lines of code that don't contribute to the program.
- Comments: code is properly commented. There should be a header comment that includes the author's name and describes the program's purpose. Additional comments in the code should help clarify how the program works.
- Organization: use whitespace to indicate the logical structure of the code.
  - Lines of code that logically belong together are close together in the file.
  - Import statements are at the very top…
  - followed by function definitions…
  - then followed by the rest of the code.

HW 03

CSC106: Assignment 3 (0.25 pts)

- due Monday 4/20/26 by class time (1:50 pm)
- Runestone Chapter 5 (see schedule for specific sub-sections)
- Practice Problems for Week 3:
  - Submit code on Gradescope,
  - Bring written answers to class (0.25 engagement points awarded on these)
  - Graded on effort/completion

starter code

- birthday.py
- criket.py

NAME:

Practice Problems for Week 3

Please complete, bring to class (written answers) and submit on gradescope (code) by end of day on **Monday 4/20/26**.

You will be submitting the following files:

- birthday.py
- cricket.py

Programming Part 1: Happy Birthday

1. Download the starter file `birthday.py` from the course website and read the instructions inside before continuing.
2. Write a function that "sings" Happy Birthday for someone. The function should take the person's name as a parameter and then print Happy Birthday in the following format.:

```
Happy Birthday to you!
Happy Birthday to you!
Happy Birthday dear <name>!
Happy Birthday to you!
```

3. Make sure to include appropriate comments.
4. Save your program using the filename `birthday.py`.

Programming Part 2: Cricket Chirping

1. Download the starter file `cricket.py` from the course website and read the instructions inside before continuing.
2. Amos Dolbaer discovered that, for certain types of crickets, there's a relationship between the temperature and the rate at which they chirp (https://en.wikipedia.org/wiki/Dolbear's_law). More specifically, the relationship is as follows:

$$T = 50 + (N - 40)/4$$

where T is the temperature in Farenheight and N the number of chirps per minute.

Write a function that calculates the temperature when given the number of times a cricket chirps in 15 seconds. Your function should do the following:

a. calculate the chirps per minute
b. calculate the temperature based on chirps per minute
c. return the temperature

3. Save your program in a file called `cricket.py`

Written Questions

1. 1. Which of the following are valid first lines for a Python function definition? Please explain why the invalid ones are not valid.

a. `def plus ten (anumber):`

b. `def double (anumber)`

c. `def greeting:`

d. `percentage(total, part):`

e. `def print_introduction():`

f. `def move()`

g. `def goto(x coordinate y coordinate):`

2. Consider the Python program below:

```
1)  def sum_to_n(n):
2)       total = 0
3)       for count in range(0, n+1):
4)            total += count
5)       return total
6)
7)  print(sum_to_n(6))
```

a. Draw a memory diagram that shows what memory looks like after the Python interpreter has executed line 2.

b. Draw a memory diagram that shows what memory looks like after the Python interpreter has executed line 4 for the first time.

c. How many times will line 4 be executed in total?

d. Draw a memory diagram that shows what memory looks like right before the Python interpreter executes line 5.

e. Draw a memory diagram that shows what memory looks like right after the Python interpreter executes line 7.

3. Print or return?

Consider the following (incomplete) function definitions.

Given the way these functions are used in the code at the end, complete the dotted lines by either turning them into a statement that **returns** the indicated value or into a statement that **prints** the indicated value.

Also complete the dotted lines in the code at the end by choosing to either `return` or `print` the value of the variable `a`.

```
def f1():
    a = int(input('Please input a number: '))

    ..................a........

def f2(var):
        squared = var**2

    ..................squared........

def f3(var):

    ..................var**3........

a = 10

..................a........
f2(a)
print(f1() + 3)
a = f3(a) * 5

..................a........
```

f. Assuming that the user input 1 when asked for a number, what will this code print to the screen?

.....

.....

.....

.....

.....

.....

4. Check-in

4.a. Were you able to complete all of the programming activities (yes/no)?

……

4.b. What, if anything, did you struggle with when working on the activities?

……

……

……

……

4.c. If you managed to overcome the challenges, how did you do it?

……

……

……

……

Weekly Notes

Week 1

Notes from Week 1

- Day 1 (Mon): Seymour Pappert Turtle Documentary
- Day 2 (Wed): Instructions for First Program Activity
- Day 3 — Lab (Thurs): Reeborg activity: newspaper

Python Examples

- in-class examples from Friday: https://drive.google.com/file/d/1UW3fMha6Ki4dDR_sidpeiV-PDeBSFEPi/view?usp=sharing
  - examples of comments, importing turtle, variables and for loops
  - also includes link to turtle library documentation

Week 2

In-class activities for week 2

- Mon: Runestone Ch. 1 review
- Wed: More Turtle Drawing
- Thurs (lab): Variables and Loops
- Fri: Tracing & Memory Diagrams

Key Resources this week

- all also found in Resources page
- Python Documentation
- Python Tutor

Examples from class

- Coding constructs: examples from class Monday - re-written a bit for clarity
- Variables and types: examples from class Thurs (lab) - in class we used PythonTutor but this is the code copied into a file in Idle

Week 3

In-class activities for week 3

- Mon: Practice with functions
  - starter code: stars0.json
  - starter code: stars1.json
- Wed: More Practice with functions & input
- Thurs (lab): Functions lab
  - starter code: house.py
- Fri: Study Questions

About

References